INTERNSHIP PROJECT TECHNICAL REPORT

Project Title: Secure Authentication System Web Application
Developer: Ankit Kumar Singh | **Branch:** B.Tech CSE
Academic Timeline: 2025-2029 | **Verification Ref:** CodeTechnology Hub

1. Executive Summary

This technical project report outlines the software design, database schemas, and cryptographic pipelines established during the development of a production-ready **Secure Authentication System**. The primary goal of the application is to demonstrate zero-trust local authentication capabilities by completely abstracting plain-text inputs from persistent storage blocks, utilizing state-of-the-art secure hashing and stateless token verification models.

2. System Workspace & Directory Blueprint

The system is isolated inside its own flat root directory framework to avoid dependencies overlapping with other adjacent software models in the repository ecosystem. The workspace layout maps directly to industrial standards as follows:

```
■ secure authentication/
■   ■■■ ■ static/
■   ■   ■■■ style.css
■   ■   ■■■ script.js
■   ■■■ ■ templates/
■   ■   ■■■ index.html (Unified Register, Login, & Profile Interface)
■   ■■■ app.py            -- Flask Application Core & REST Routing Matrix
■   ■■■ requirements.txt   -- Declared Project Dependency Manifest
■   ■■■ .gitignore        -- Private/Runtime Sandbox Exclusion Rules
■   ■■■ users.db          -- Persistent SQLite Database Instance
```

3. Cryptographic Infrastructures (Bcrypt & PyJWT)

3.1 Cryptographic Password Salting (Bcrypt): In accordance with security constraints, user passwords are treated as compromised vectors if stored raw. The registration endpoint utilizes the blowfish cipher hashing architecture provided by *Flask-Bcrypt*. The engine dynamically introduces a unique random string (salt) to every incoming password block before running it through exhaustive calculations, generating a distinct irreversible hash string.

3.2 Stateless Token Verification (PyJWT): Traditional session architectures demand continuous lookups into persistent databases. This project transitions tracking overhead to client contexts via JSON Web Tokens. Upon successful verification of password hashes, the backend signs a token packed with user parameters and an automatic **2-hour expiration time window** using private environment secret variables.

4. REST API Endpoint Infrastructure

The application presents standard decoupled JSON access paths for web interfaces or third-party client integrations. The table below represents the exact route configurations:

HTTP Method	Endpoint Route	Verification Rules & Actions
POST	/api/register	Validates parameters; salts and hashes password strings, inserts user record into users.db.
POST	/api/login	Queries email records, runs Bcrypt check loops, signs and issues 2-hour expiring JWT access token.
GET	/api/profile	Demands valid HTTP Authorization headers containing Bearer JWT tokens; blocks expired entries.

5. Environment Management & Git Hygiene

To guarantee standard clean execution guidelines for evaluations, private context assets are separated via *python-dotenv*. Critical security factors, including private verification signature keys, are kept inside isolated local files. The `.gitignore` manifest handles masking to prevent deployment tracking engines from logging transient runtime caches, compiled pycache blocks, or persistent user database entries (`*.db`).

6. Project Submission Matrix

All code assets, implementation specifications, and presentation guides are packed into a unified workspace. Evaluators can confirm branch management strategies and cross-verify project details via the main repository portal link below:

> **Official Code Submission Repository:**
> https://github.com/ankitsingh085432-afk/2025-2029_Ankitkumarsingh_4110_3sme_2cse13-